CVA6 RISC-V

Trace Profiling Eclipse Plugin PROJECT 6

Integrating E-Trace & System Trace Decoders · Real-Time Profiling · Eclipse IDE

Final Project Demonstration · Iqra Jawad Ahmed

What Is This Project?

The Problem

Hardware trace data from CVA6 RISC-V is encoded and unusable by developers without a decoder and profiler built into their IDE.

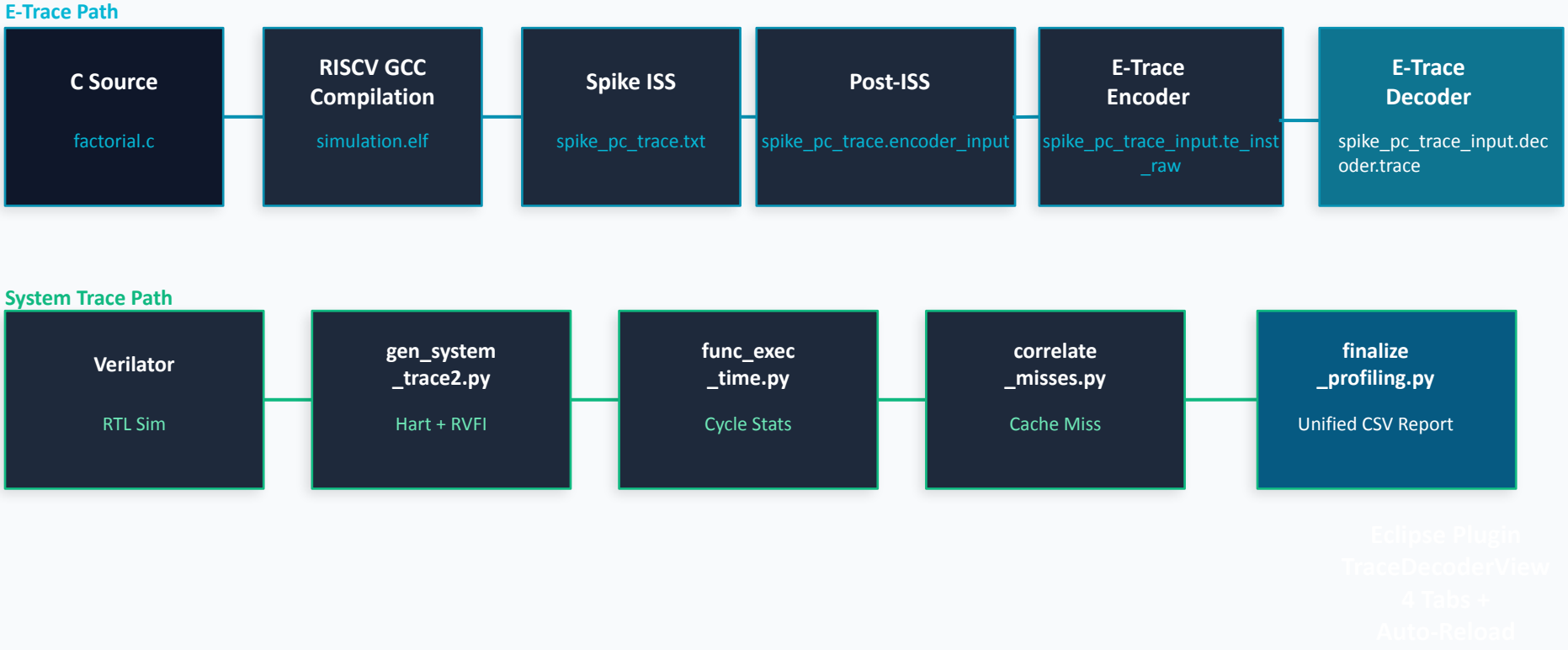
Our Solution

An Eclipse plugin that reads raw encoded trace, decodes it live (E-Trace + System Trace), correlates with the ELF binary, and generates profiling reports.

Who Benefits

Embedded firmware engineers who need cycle-accurate profiling, function hotspots, cache misses, memory access patterns; without leaving Eclipse.

End-to-End Pipeline Architecture



Checkpoint 1: E-Trace Decoder

What the Encoder/Decoder Does

- Reads spike_pc_trace.txt: the raw PC trace from Spike ISS simulation
- Post-ISS converts it to encoder_input format per RISC-V trace spec
- Encoder compresses the PC stream into te_inst_raw packets
- Decoder reconstructs the original instruction stream
- Diff check verifies decode accuracy, any mismatch is reported

Checkpoint 1: System Trace Decoder (gen_system_trace2.py)

Inputs

› trace_hart_0.dasm: hart cycle/PC log

› trace_rvfi_hart_00.dasm: register & memory deltas

› simulation.elf: symbol table for function names

Processing

› Parses HART_RE cycle + PC per instruction

› Matches RVFI register deltas + memory ops

› Binary search maps each PC → function name

› Classifies: EXEC / LOAD / STORE / BRANCH / CALL / RET

Output

› system_trace.txt: one row per instruction

› Columns: CYCLE · PC · MNEMONIC · EVENT · FUNC · REG_DELTA · MEM_ACCESS

› Noise functions & htif loops collapsed for readability

Checkpoint 2: Profiling Engine

func_exec_time.py

→ function_exec_time.csv

- Self cycles per function
- CPI (cycles per instruction)
- Avg cycles per call
- LOAD / STORE / BRANCH counts
- Memory intensity score

correlate_misses.py

→ rtl_profiling_report.csv

- I-Cache miss count per function
- D-Cache miss count per function
- Total cache misses
- PC-to-function via binary search

finalize_profiling.py

→ unified_profiling_report.csv

- Merges ALL three sources
- ELF addr2line for PC ranges
- Source file + line per function
- Eclipse-ready unified CSV + TXT

Eclipse Plugin: TraceDecoderView

Tab 1: Unified Profiling

12-column table: Function · PC Start/End · Source Location · Total Cycles · % of Program · Calls · Avg Cyc/Call · Instr Count · I$ Misses · D$ Misses · Total Cache Misses

Tab 2: System Trace

First 5000 instructions shown. Columns: Cycle · PC · Mnemonic · Event · Function · Reg Delta · Mem Access. Parsed from system_trace.txt.

Tab 3: E-Trace

First 5000 decoded PC addresses from spike_pc_trace_input.decoder.trace. Row number + 0x-prefixed address.

Tab 4: Cache Misses

RTL profiling data: Function · I$ Misses · D$ Misses · Total. Sourced from rtl_profiling_report.csv.

Live Demo — What You'll See

01

Run the Pipeline

Execute run_trace_pipeline1.sh: watch 11 steps execute: compile → Spike → Verilator → E-Trace encode/decode → System Trace → Profiling

02

E-Trace Verified

Terminal shows '[OK] E-Trace MATCH': diff between spike_check.txt and decoder_check.txt is zero

03

Profiling CSV Generated

function_exec_time.csv shows per-function cycles, CPI, memory intensity which is used by unified_profiling_report.csv to merge everything.

04

Eclipse Auto-Loads

TraceDecoderView detects pipeline_done.trigger, reloads all 4 tabs — live profiling data visible inside Eclipse with no clicks

Learning Outcomes Achieved

🔌 Eclipse RCP/CDT Plugin Dev

Built a complete ViewPart with SWT tables, CTabFolder, GridLayout from scratch. Integrated plugin into a live Eclipse RCP runtime instance.

🔍 Hardware Trace Profiling

Deep understanding of E-Trace (RISC-V trace spec) encoder/decoder flow, Post-ISS format, and how to turn encoded packets back into a verified instruction trace.

📊 Data Visualization & Reporting

Multi-source data fusion: cycle stats + cache misses + ELF symbols ⟶ unified CSV + human-readable summary with hotspot rankings, displayed live in Eclipse.

Project Complete ✓

Both checkpoints delivered on time.
End-to-end from C source to Eclipse profiling table.

- 11-step automated pipeline (one shell script)
- E-Trace encoder + decoder with diff verification
- System Trace: cycle-accurate per-instruction merged trace
- 3 profiling scripts for accurate timing, cache misses, unified reporting
- Eclipse TraceDecoderView: 4 tabs + trigger-file auto-reload